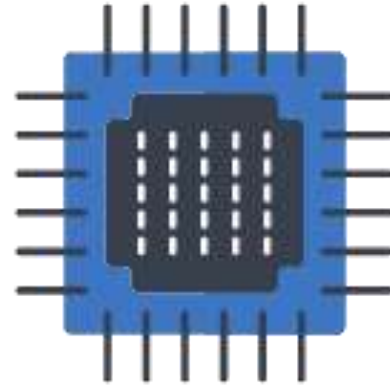


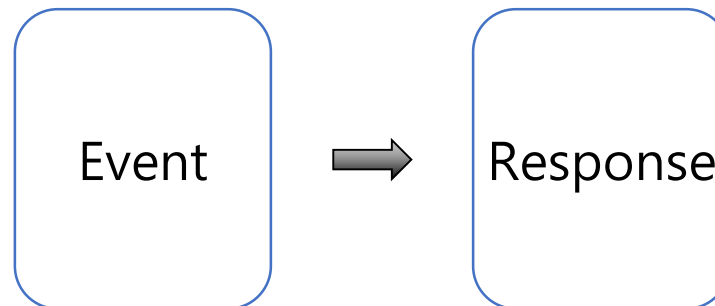


MICROPROCESSOR -LAB

Lab8: Interrupts

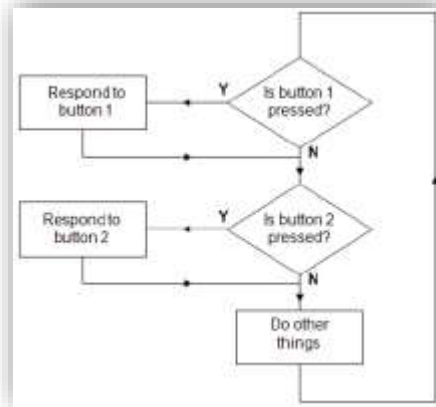
Event-Triggered Response (1)

- Event-Triggered Response
 - Event: A specific change in state within a system or an external input
 - For instance, 'a button is pressed' or 'a sensor value exceeds a certain threshold'
 - Response: A predefined action performed when the specific event is detected
 - For instance, 'turn on an LED'

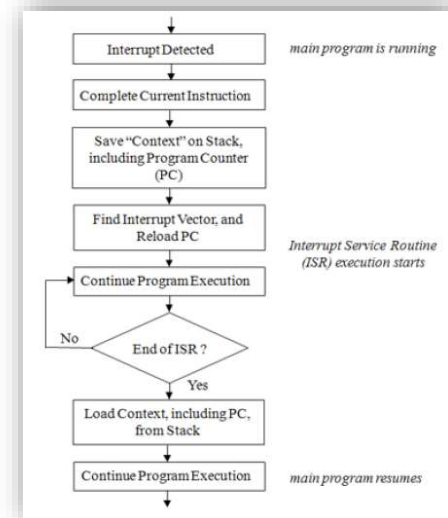


Event-Triggered Response (2)

- There are two primary methods for handling Event-Triggered Response: Polling and Interrupt
 - Polling: A method where the CPU continuously checks the status of an event at regular intervals
 - It is straightforward to implement but inefficient (waste processing power even when no events occur)
 - If an event occurs between polling intervals, the system may not respond
 - Interrupt: A method where the CPU is notified asynchronously when an event occurs
 - The CPU can perform other tasks until an event occurs, improving overall efficiency
 - This allows the system to respond immediately without continuously checking the status



<polling>



<interrupt>

Lab 1-1. Detect Button Press While Counting Up (POLLING)

```
1 #include "mbed.h"
2
3 DigitalIn button(PA_14);
4 Serial pc(USBTX, USBRX);
5 DigitalOut myled(LED1);
6
7 int main() {
8     int i = 0;
9     while(1) {
10         if(!button) {
11             pc.printf("Button Press Detected!\r\n");
12             myled = !myled;
13         }
14         i++;
15         pc.printf("%d\r\n", i);
16         wait(1);
17     }
18 }
```

- Run this code and check if it works as intended
- When you press the button, the system may not be checking the button's state at that exact moment
- Because it only checks the state once every second
- This issue (related to polling frequency) is indeed some of the key drawbacks of the polling method in event handling

Lab 1-2. Detect Button Press While Counting Up (INTERRUPT)

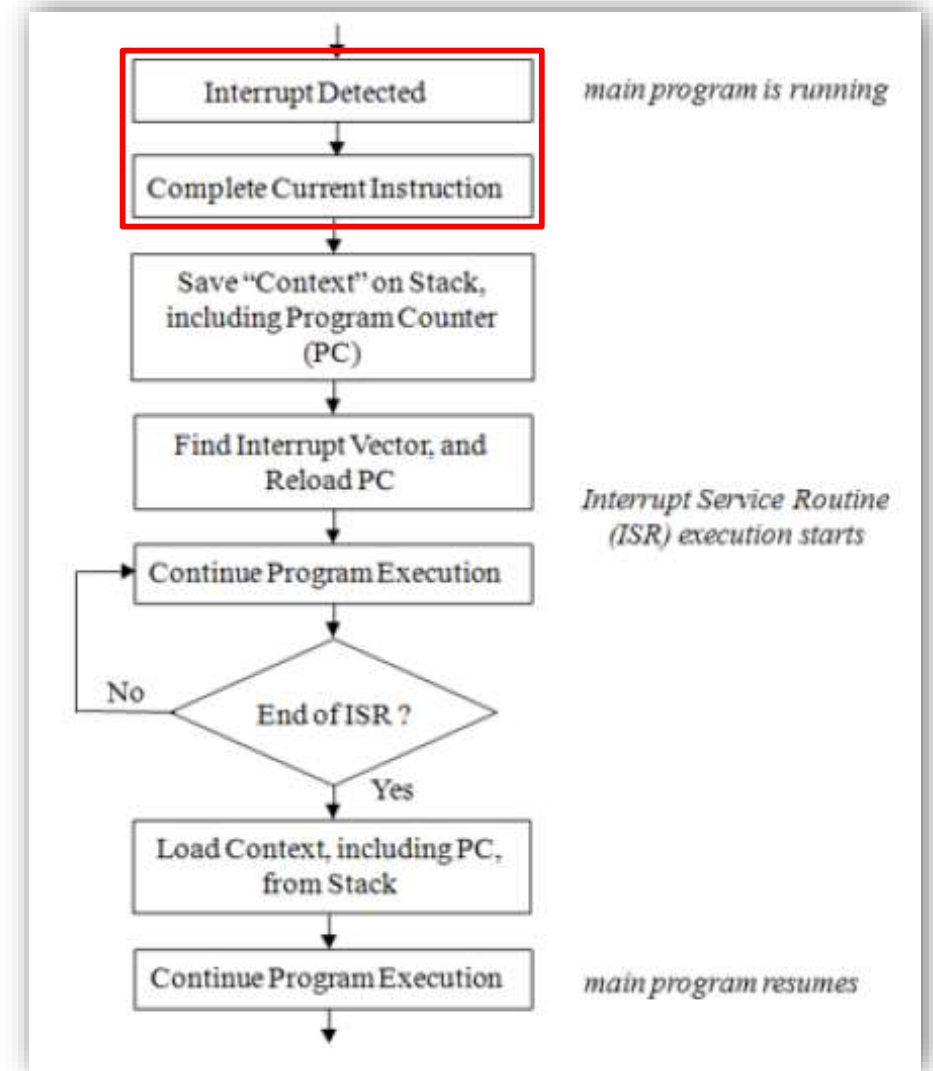
```
1 #include "mbed.h"
2
3 InterruptIn button(PA_14);
4 Serial pc(USBTX, USBRX);
5 DigitalOut myled(LED1);
6
7 void ISR1() {
8     pc.printf("Button Press Detected!\r\n");
9     myled = !myled;
10    wait(0.2);
11 }
12
13 int main() {
14     button.fall(&ISR1);
15     for(int i=0; ; i++) {
16         pc.printf("%d\r\n", i);
17         wait(1);
18     }
19 }
```

- PA14 pin of the microcontroller is declared as an interrupt pin
- It attaches the ISR1 function to the falling edge of the button press
- This means that when the button is pressed, the ISR1 function will be called
- ❖ ISR (Interrupt service routine) indicates a special function in embedded systems programming that is executed in response to an interrupt signal
- InterruptIn APIs

Function	Description
rise(&ISR)	attaches a ISR to be called when the signal rises
fall(&ISR)	attaches a ISR to be called when the signal falls
enable()	enables the interrupt, allowing the attached ISR to be called
disable()	disables the interrupt, preventing the attached ISR from being called

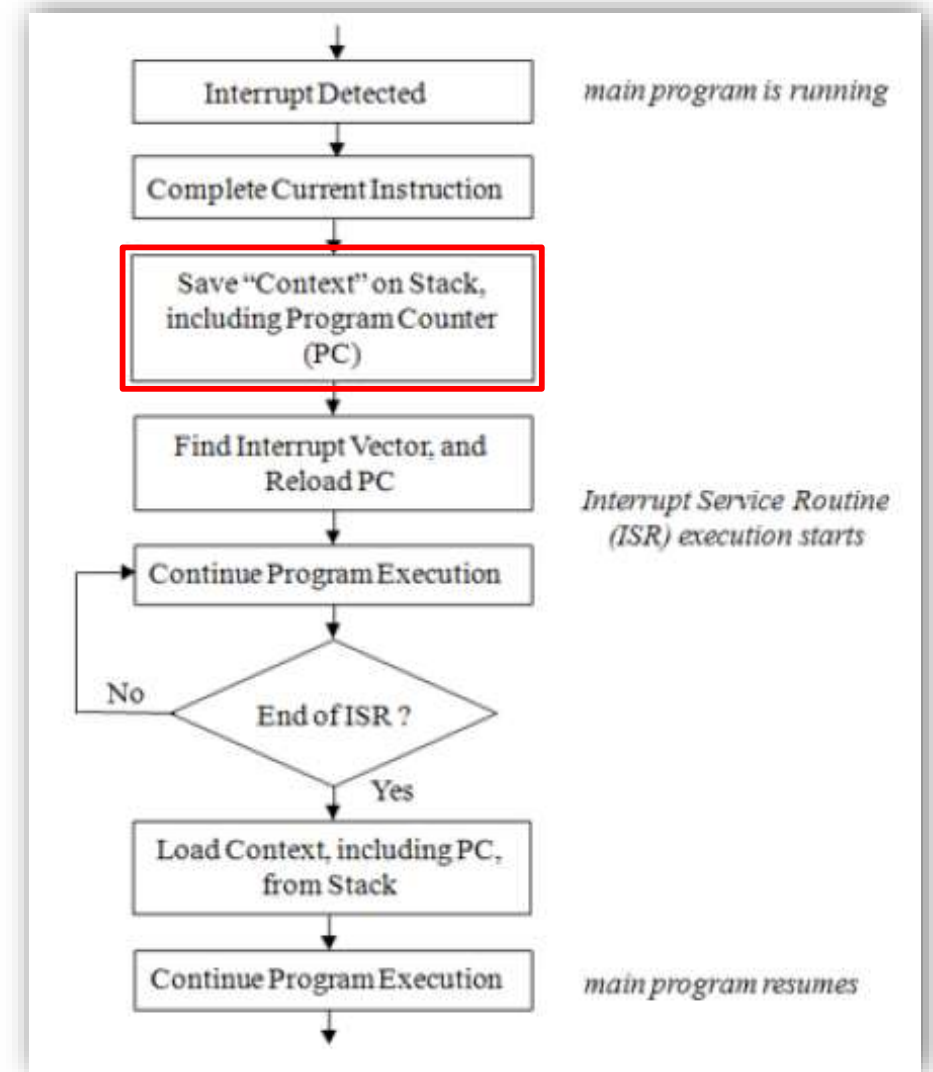
The Detail Process of Interrupt (1)

- When an external event occurs (such as a button press), it triggers an interrupt
- This action sets an internal flag that indicates that an interrupt has occurred.
 - Internal flag is a specific bit in the CPU's status register
 - The CPU periodically checks the status of this interrupt flag
 - If the flag is set, the CPU acknowledges the interrupt request (which may involve clearing the interrupt flag and prioritizing the interrupt request based on its importance)



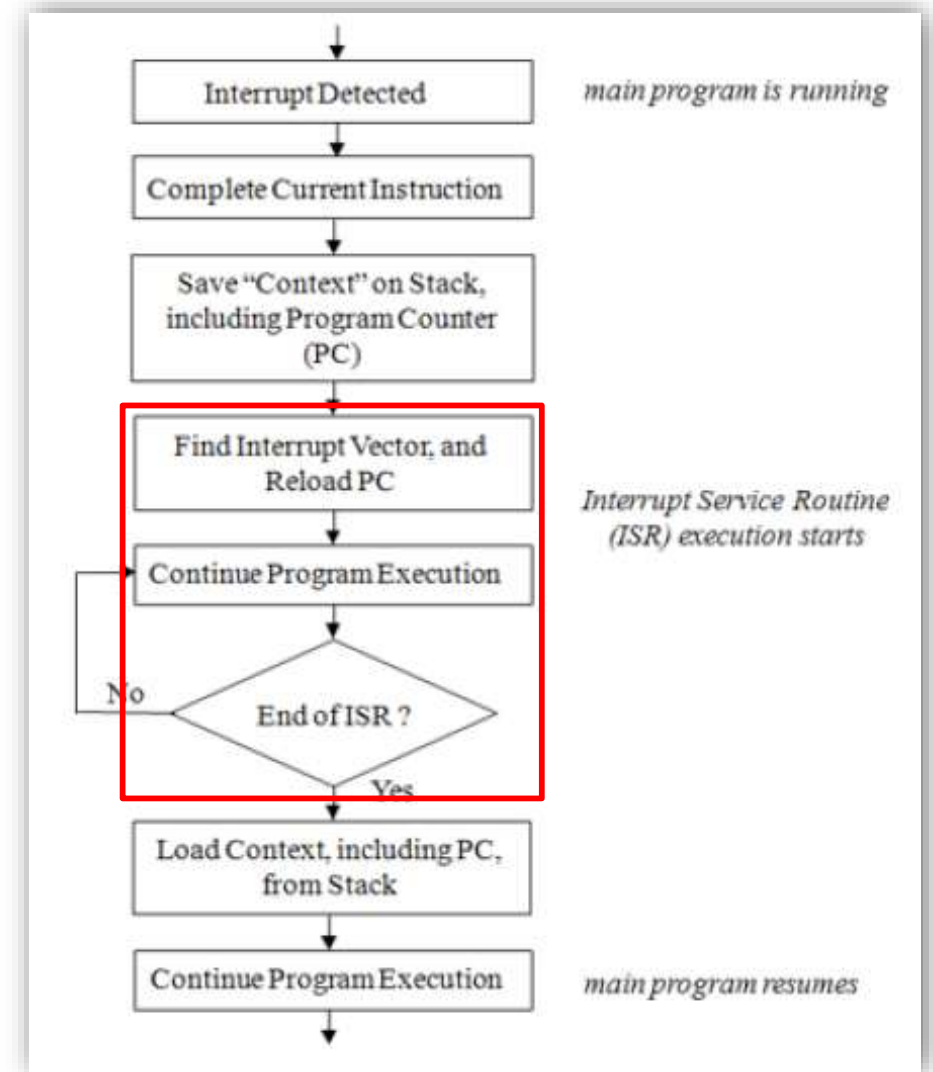
The Detail Process of Interrupt (2)

- Before the CPU branches to the ISR, it must save the current context to a small memory block called the stack
 - Program Counter (PC): The address of the next instruction to execute in the main program
 - CPU Status Register: Flags and states that indicate the current state of the CPU
 - General Purpose Registers: Any other relevant registers that contain data the main program may need after returning from the ISR



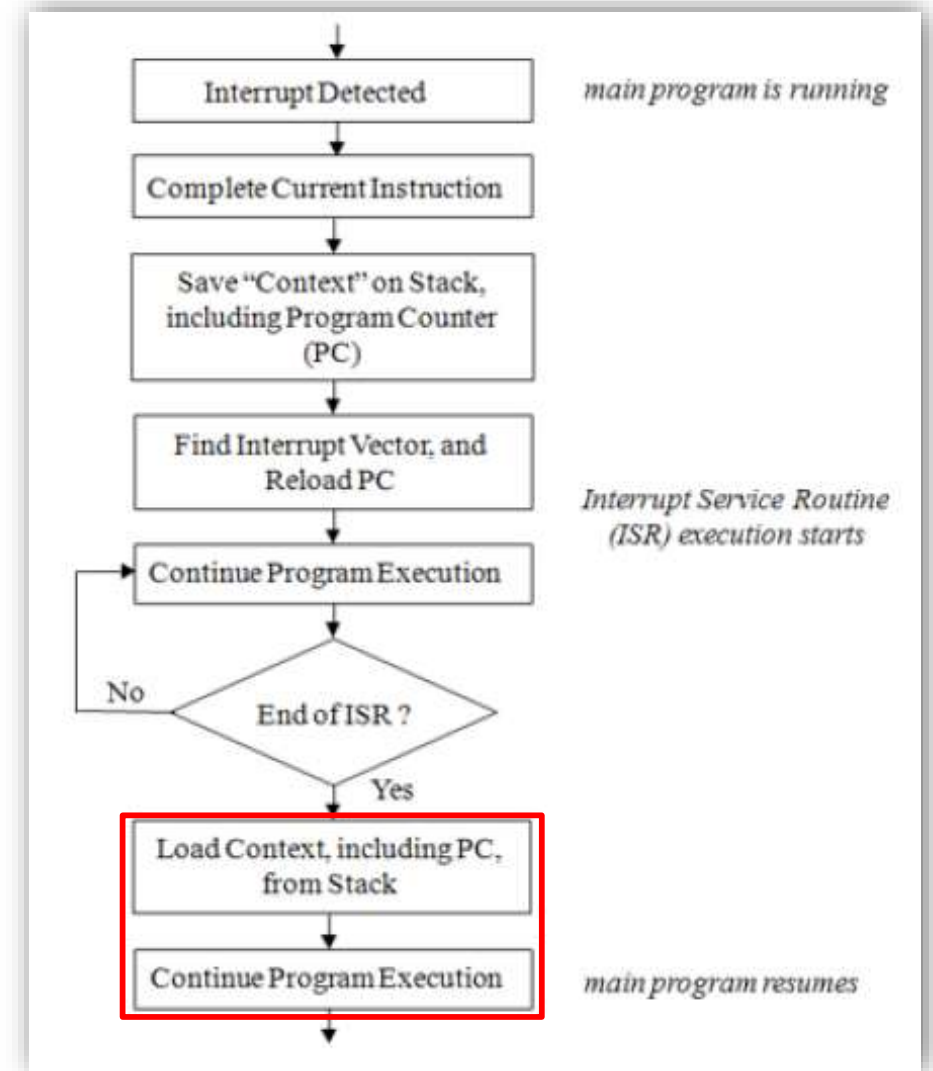
The Detail Process of Interrupt (3)

- The CPU loads the address of the corresponding ISR from the IVT into the program counter and executes the ISR
 - The IVT (Interrupt Vector Table) is a predefined memory location that maps different interrupts to their respective ISRs



The Detail Process of Interrupt (4)

- After the ISR is completed, the CPU returns to the main code immediately
 - by restoring the previously saved values from the stack into the program counter and registers
- Some interrupts can be temporarily disabled, meaning the CPU ignores them while performing critical operations
 - This ensures that the CPU can safely complete the current operation without being disrupted by other events



Assignment. Motor Control Using Button

- Write a code to control a motor using button presses
 - Please utilize the motordriver.h and motordriver.cpp files that you created in the previous lab class
 - The program should initialize three buttons to adjust the motor's direction and speed
 - Each button should perform the following actions:
 - Button 1: Change the rotation direction of the motor
 - Button 2: Increase the speed of the motor by 0.1, up to a maximum speed of 1.0
 - Button 3: Decrease the speed of the motor by 0.1, down to a minimum speed of 0.2
 - Each button press should have a debounce mechanism to prevent multiple triggers from a single press

Assignment. Motor Control Using Button

- Skeleton Code

```
1 #include "mbed.h"
2 #include "motordriver.h"
3
4 InterruptIn button1(PA_14);
5 InterruptIn button2(PB_7);
6 InterruptIn button3(PC_4);
7 Serial pc(USBTX, USBRX);
8 Motor A(D11, PC_8);
9
10 int direction = 1;
11 float speed = 0.1;
```

```
1 void print_current_state() {
2     pc.printf("Current State:\r\n");
3     pc.printf("direction: %d\r\n", direction);
4     pc.printf("speed: %.1f\r\n", speed);
5 }
6
7 void change_direction() {
8     direction = !direction;
9     pc.printf("Change Rotation Direction to %d\r\n", direction);
10    print_current_state();
11    wait(0.2);
12 }
13
14 void acc() {
15     if(speed < 1) {
16         speed up
17         pc.printf("Increase Speed to %.1f\r\n", speed);
18     } else {
19         pc.printf("Reached MAX Speed (No Increase)\r\n");
20     }
21     print_current_state();
22     debounce (0.2 sec)
23 }
24
25 void dec() {
26     if(speed > 0.2) {
27         speed up
28         pc.printf("Decrease Speed to %.1f\r\n", speed);
29     } else {
30         pc.printf("Reached MIN Speed (No Decrease)\r\n");
31     }
32     print_current_state();
33     debounce (0.2 sec)
34 }
```

```
1 int main() {
2     print_current_state();
3
4
5
6     A.forward(speed);
7     while(1) {
8         if(direction == 1) {
9             A.forward(speed);
10        } else{
11            rotate backward
12        }
13    }
14 }
```

- Include the necessary libraries for Mbed and the motor driver
- Initialize three interrupt pins for the buttons (button1, button2, button3)
- Initialize a serial connection
- Initialize a motor object (Motor A)
- Declare a variable to control the motor's direction (e.g., int direction = 1;)
- Declare a variable to control the motor's speed (e.g., float speed = 0.1;).

Assignment. Motor Control Using Button

- Skeleton Code

```
1 #include "mbed.h"
2 #include "motordriver.h"
3
4 InterruptIn button1(PA_14);
5 InterruptIn button2(PB_7);
6 InterruptIn button3(PC_4);
7 Serial pc(USBTX, USBRX);
8 Motor A(D11, PC_8);
9
10 int direction = 1;
11 float speed = 0.1;
```

```
1 void print_current_state() {
2     pc.printf("Current State:\r\n");
3     pc.printf("direction: %d\r\n", direction);
4     pc.printf("speed: %3.1f\r\n", speed);
5 }
6
7 void change_direction() {
8     direction = !direction;
9     pc.printf("Change Rotation Direction to %d\r\n", direction);
10    print_current_state();
11    wait(0.2);
12 }
13
14 void acc() {
15     if(speed < 1) {
16         [ ] speed up
17         pc.printf("Increase Speed to %3.1f\r\n", speed);
18     } else {
19         pc.printf("Reached MAX Speed (No Increase)\r\n");
20     }
21     print_current_state();
22     [ ] debounce (0.2 sec)
23 }
24
25 void dec() {
26     if(speed > 0.2) {
27         [ ] speed up
28         pc.printf("Decrease Speed to %3.1f\r\n", speed);
29     } else {
30         pc.printf("Reached MIN Speed (No Decrease)\r\n");
31     }
32     print_current_state();
33     [ ] debounce (0.2 sec)
34 }
```

```
1 int main() {
2     print_current_state();
3     [ ]
4     [ ]
5     [ ]
6     A.forward(speed);
7     while(1) {
8         if(direction == 1) {
9             A.forward(speed);
10        } else{
11            [ ]
12        } rotate backward
13    }
14 }
```

- Declare a function that prints the current direction and speed
- Declare an ISR for button1 that changes the motor's rotation direction
- Declare an ISR for button2 that increases the motor's speed by 0.1 (up to a maximum of 1.0)
- Declare an ISR for button3 that decreases the motor's speed by 0.1 (down to a minimum of 0.2)

Assignment. Motor Control Using Button

- Skeleton Code

```
1 #include "mbed.h"
2 #include "motordriver.h"
3
4 InterruptIn button1(PA_14);
5 InterruptIn button2(PB_7);
6 InterruptIn button3(PC_4);
7 Serial pc(USBTX, USBRX);
8 Motor A(D11, PC_8);
9
10 int direction = 1;
11 float speed = 0.1;
```

```
1 void print_current_state() {
2     pc.printf("Current State:\r\n");
3     pc.printf("direction: %d\r\n", direction);
4     pc.printf("speed: %3.1f\r\n", speed);
5 }
6
7 void change_direction() {
8     direction = !direction;
9     pc.printf("Change Rotation Direction to %d\r\n", direction);
10    print_current_state();
11    wait(0.2);
12 }
13
14 void acc() {
15     if(speed < 1) {
16         [ ] speed up
17         pc.printf("Increase Speed to %3.1f\r\n", speed);
18     } else {
19         pc.printf("Reached MAX Speed (No Increase)\r\n");
20     }
21     print_current_state();
22     [ ] debounce (0.2 sec)
23 }
24
25 void dec() {
26     if(speed > 0.2) {
27         [ ] speed up
28         pc.printf("Decrease Speed to %3.1f\r\n", speed);
29     } else {
30         pc.printf("Reached MIN Speed (No Decrease)\r\n");
31     }
32     print_current_state();
33     [ ] debounce (0.2 sec)
34 }
```

```
1 int main() {
2     print_current_state();
3     [ ]
4     [ ]
5     [ ]
6     A.forward(speed);
7     while(1) {
8         if(direction == 1) {
9             A.forward(speed);
10        } else{
11            [ ]
12        } rotate backward
13    }
14 }
```

- Attach the change_direction(), acc(), and dec() functions to their respective button presses using the falling edge trigger